Autor:
Ronie Martínez

Librerías básicas de Python para manipulación y visualización de datos

Introducción

La manipulación y visualización de datos son aspectos cruciales en el campo de la ciencia de datos y la programación en general. Python, un lenguaje de programación ampliamente utilizado, ha ganado una gran popularidad en este ámbito gracias a la riqueza y variedad de sus librerías, y su facilidad de uso en cuanto a simplicidad y escalabilidad. Python se ha convertido en un lenguaje de referencia en el campo de la ciencia de datos debido a su sintaxis clara y su amplio ecosistema de librerías diseñadas específicamente para tareas relacionadas con el manejo y análisis de datos. En particular, Python sobresale en la manipulación de datos tabulares y la creación de visualizaciones informativas.

En esta lectura de profundización se explorarán las librerías básicas de Python que son fundamentales para manipular y visualizar datos de manera eficiente. Estas herramientas son esenciales para cualquier profesional o entusiasta que desee trabajar con datos de manera efectiva. Se definirán y ampliarán cuatro de las librerías más fundamentales de Python para estas tareas: NumPy y Pandas para la manpulación de datos homogéneos y heterogéneos, y Matplotlib y Seaborn para la creación de gráficos como presentación visual de datos y resultados, lo que permite a los profesionales de datos abordar una amplia variedad de desafíos en sus proyectos.

La librería principal de Python para la computación científica es NumPy. Ayuda a trabajar con arreglos y matrices multidimensionales, que son necesarios para realizar operaciones matemáticas y estadísticas en datos. Se pueden realizar tareas como agregar, filtrar y manipular datos con esta librería. Además, proporciona una amplia variedad de funciones matemáticas que son esenciales para el análisis de datos. Muchas otras librerías y herramientas de ciencia de datos basadas en Python se construyen sobre esta como base.

Pandas (Chen, 2017) es una excelente herramienta para manipular y analizar datos tabulares etiquetados. Se pueden representar y trabajar con los datos en

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

forma de tablas, similares al manejo de una hoja de cálculo de Excel. Con Pandas, se pueden realizar operaciones de agrupación y resumen, limpieza y preprocesamiento de datos, consultas, selección de columnas y filas y mucho más. La versatilidad de Pandas lo convierte en una herramienta esencial para cualquier analista de datos o científico de datos.

Matplotlib y Seaborn son librerías de visualización de datos que permiten crear una amplia variedad de gráficos y figuras de manera programática. Desde gráficos de barras e histogramas, hasta diagramas de dispersión y gráficos de líneas, Matplotlib proporciona las herramientas necesarias para representar datos de manera efectiva y comprensible. Se puede personalizar la apariencia de los gráficos, agregar etiquetas, títulos y leyendas, y guardar las visualizaciones en diferentes formatos. La capacidad de crear visualizaciones atractivas y significativas es esencial para comunicar resultados y descubrimientos en proyectos de ciencia de datos (Bisong, 2019).

Manipulación y visualización de datos

El almacenamiento y manipulación de los datos en Python es importante, ya que, aunque se tenga libertad y flexibilidad con respecto a la no definición previa de estos, se debe ser consciente de cómo es interpretada esta información. De esta manera, una variable de tipo *int*, por ejemplo, es un objeto que contiene información de cómo se maneja la memoria correspondiente, el tipo, el tamaño y el valor.

La siguiente estructura básica en Python es la lista (*list*), que es una estructura que contiene varios objetos, incluso de diferentes tipos, como lógicos (*boolean*), texto (*string*), decimales (*float*) y enteros (*int*):

L = [True, "2", 3.0, 4]

Como se presentó en el tema principal, las librerías de Python proporcionan un gran apoyo y brindan gran versatilidad para tareas de manejo de datos. El comando para añadir estas librerías a nuestro ambiente de trabajo es **import** seguido del nombre de la librería. Adicionalmente, se pueden usar o importar métodos y funciones específicas de cierta librería, siguiendo el formato tradicional **[nombre_de_la_librería].[método]**; se puede usar la interactividad de IPython para revisar los métodos disponibles y también para auto completado, esto mediante la tecla **<TAB>.** Finalmente, se puede asignar un

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

alias corto con el comando as, tanto para facilidad de escritura como para mantener la convención dentro de la comunidad (VanderPlas, 2016).

NumPy

Como convención se suele utilizar la librería NumPy (Numpy, 2022) con el nombre corto **np**:

import numpy as np

Para crear un arreglo (*array*) se utiliza el método del mismo nombre, y el tipo de dato se puede definir de manera explícita, escribiéndolo como parámetro, o de manera implícita, es decir, sin especificarlo. En este último caso, NumPy se ajusta automáticamente (de ser posible) al tipo de dato que englobe todos los elementos:

np.array([3.14, 4, 2, 3] , dtype='float32')

De igual manera, estos arreglos pueden ser multidimensionales. El siguiente ejemplo es la representación de un arreglo bidimensional de 3x3, es decir, una matriz:

np.array([range(i, i + 3) for i in [2, 4, 6]])

Existen también varios métodos para crear arreglos de manera más eficiente, y algunos ejemplos se presentan en la Tabla 1.

TABLA 1. EJEMPLOS DE COMANDOS Y SUS RESULTADOS UTILIZANDO MÉTODOS DE NUMPY

Comando	Resultado
np.zeros(10, dtype=int)	Arreglo de ceros de longitud 10
np.ones((3, 5), dtype=float)	Arreglo de unos de dimensión 3x5
np.full((3, 5), 3.14)	Arreglo lleno de 3.14 de dimensión 3x5
np.arange(0, 20, 2)	Arreglo de una secuencia lineal que inicia en 0 y termina en 20, a pasos de 2
np.linspace(0, 1, 5)	Arreglo de 5 valores equidistantes entre 0 y 1
np.random.random((3, 3))	Arreglo con valores aleatorios entre 0 y 1 (distribución uniforme) de 3x3
np.random.normal(0, 1, (3, 3))	Arreglo con valores aleatorios entre 0 y 1 (distribución normal) de 3x3
np.random.randint(0, 10, (3, 3))	Arreglo con enteros aleatorios entre 0 y 10 de 3x3
np.eye(3)	Matriz identidad de 3x3

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Los objetos de tipo array tienen varios atributos, como tamaño, dimensiones, consumo de memoria y tipo. También, están indexados numéricamente, y pueden ser particionados, redimensionados, unidos o divididos.

La indexación comienza en 0 y se pueden usar números negativos para acceder a los elementos en orden inverso (de final a inicio) y también rangos definidos mediante ':'. Para arreglos multidimensionales se puede acceder usando tuplas separadas por coma. Se pueden modificar los valores de los elementos accedidos, sin embargo, se debe considerar que el arreglo al momento de ser definido tiene un tipo fijo de datos, por lo que los nuevos valores se truncaran (de ser posible) al tipo definido inicialmente.

Una especial consideración es que cuando se extraen elementos parciales de un arreglo original, y estos valores (arreglo parcial) se modifican, estas modificaciones también afectan al arreglo original. Es por eso que, dependiendo de la utilidad, hay ocasiones en que conviene crear una copia para su posterior manipulación:

```
x = np.random.randint(10, size=(3, 4))

x_sub_copy = x[:2, :2].copy()
```

También existen ocasiones en las que es necesario redimensionar algún arreglo:

```
grid = np.arange(1, 10).reshape((3, 3))
```

Muchas de estas veces, es simplemente necesario añadir una dimensión al arreglo original, lo que se puede lograr con el comando anterior o con el siguiente:

```
grid[np.newaxis, :]
```

Mientras que para concatenar se tienen varias opciones: **np.concatenate**, **np.vstack**, y **np.hstack**, al igual que para dividir: **np.split**, **np.hsplit**, y **np.vsplit**.

Las operaciones aritméticas y las UFuncs disponibles para arreglos se presentan en la Tabla 2, y las funciones de agregación en la Tabla 3 (VanderPlas, 2016).

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

TABLA 2. OPERADORES ARITMÉTICOS IMPLEMENTADOS EN NUMPY

Operador	UFunc equivalente
+	np.add
-	np.subtract
-	np.negative
*	np.multiply
/	np.divide
//	np.floor_divide
**	np.power
%	np.mod

TABLA 3. FUNCIONES DE AGREGACIÓN DISPONIBLES EN NUMPY

Nombre	Versión NaN-safe	Descripción
np.sum	np.nansum	Suma
np.prod	np.nanprod	Producto
np.mean	np.nanmean	Promedio
np.std	np.nanstd	Desviación estándar
np.var	np.nanvar	Varianza
np.min	np.nanmin	Valor mínimo
np.max	np.nanmax	Valor máximo
np.argmin	np.nanargmin	Índice del valor mínimo
np.argmax	np.nanargmax	Índice del valor máximo
np.median	np.nanmedian	Mediana
np.percentile	np.nanpercentile	Percentiles (estadística)
np.any	N/A	Evalúa si algún elemento cumple
np.all	N/A	Evalúa si todos los elementos cumplen
np.abs	N/A	Valor absoluto

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Pandas

Mientras NumPy sirve para la manipulación de datos homogéneos basados en arreglos (array), Pandas sirve para manipular datos heterogéneos y etiquetados. Como convención se suele utilizar la librería Pandas (Pandas, 2023) con el nombre corto **pd**:

import pandas as pd

Se puede crear un objeto Series (unidimensional) a partir de una lista o arreglo:

data = pd.Series([0.25, 0.5, 0.75, 1.0])

Se puede acceder a sus elementos mediante el método **values** y a los índices mediante el método **index** o **keys()**. Estos índices no son necesariamente secuenciales ni numéricos. De esta manera, se pueden considerar como diccionarios especializados con clave (*key*) y valor (*value*), en donde la clave es el indexado por defecto:

```
population_dict = {'California': 38332521,
                   'Texas': 26448193,
                   'New York': 19651127,
                   'Florida': 19552860,
                   'Illinois': 12882135}
population = pd.Series(population_dict)
```

Para arreglos bidimensionales, se crean objetos DataFrame, y se pueden considerar a estos como una secuencia de objetos Series que comparten el mismo índice:

```
area_dict = {'California': 423967,
             'Texas': 695662,
             'New York': 141297,
             'Florida': 170312,
             'Illinois': 149995}
```

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

area = pd.Series(area_dict)

states = pd.DataFrame({'population': population,

'area': area})

También se pueden crear DataFrames a partir de listas de diccionarios, y si existen elementos faltantes Pandas las rellena automáticamente con **NaN** (*not a number*) y se ajusta a los datos y estructuras disponibles. **NaN no** es igual que 0. Existen algunos métodos para manejar estos datos faltantes, y serán revisados en posteriores contenidos. A continuación, un ejemplo de un DataFrame con datos faltantes, creado a partir de una lista de diccionarios:

pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])

Ahora, adicionalmente a los elementos e índices, se puede acceder al atributo **columns**, que también es un indexado, pero en la otra dimensión. Es necesario recordar que estos índices no pueden ser modificados de la manera tradicional (asignando un valor con '=‘). También se pueden crear máscaras con operadores condicionales **df[df > 0]** e incluso usar esos operadores condicionales en conjunto **df[(df > 0.3) & (df < 0.8)]**.

En cuanto al indexado, existen métodos para indexados "especiales":

loc: referencia el índice explícito, basado en la etiqueta.

iloc: referencia el índice implícito, basado en la posición numérica.

ix: híbrido entre ambas.

Para concatenado también existen un método de Pandas: **pd.concat()** y tiene una sintaxis similar que su homónimo en NumPy **np.concatenate()**, pero con funcionalidades extras. Dentro de los parámetros que se pueden definir en este método está join que permite definir qué tipo de intersección deseamos aplicar a las tablas (DataFrames). También está el método **pd.append()** que funciona de manera similar. Mientras que el método **pd.merge()** combina de una manera más inteligente los DataFrames y no depende de los índices de filas solamente, sino también de los índices de columnas.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Al igual que NumPy existen métodos de agregación, y se listan en la Tabla 4.

TABLA 4. MÉTODOS DE AGREGACIÓN DE PANDAS

Aggregation	Descripción
count()	Número de ítems
first(), last()	Primer y último ítem
mean(), median()	Media y mediana
min(), max()	Mínimo y Máximo
std(), var()	Desviación estándar y varianza
mad()	Desviación absoluta media
prod()	Producto de todos los ítems
sum()	Suma de todos los ítems
describe()	Resumen de algunas de las estadísticas básicas anteriores

Generalmente hay necesidad de dividir (*split*), aplicar (*apply*) o combinar (*combine*) ciertas porciones del DataFrame, de lo cual se encarga el método **GroupBy()** en combinación con alguna función, generalmente de agregación. Para procesos más complejos se tiene el método **pivot_table()**. Un ejemplo de ambas implementaciones sobre el conjunto de datos del *titanic* (*dataset* muy popular en el entorno de aprendizaje) se presenta a continuación:

```
import seaborn as sns

titanic = sns.load_dataset('titanic')

titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()

titanic.pivot_table('survived', index='sex', columns='class')
```

Ambas implementaciones obtienen el mismo resultado que se presenta en la Ilustración 1. Esta manipulación de datos presenta una mejor idea de cómo el género y la clase afectó a la supervivencia, información presentada mediante un índice.

ILUSTRACIÓN 1. ÍNDICE DE SUPERVIVENCIA EN EL DATASET DEL TITANIC, EN FUNCIÓN DEL GÉNERO Y CLASE.

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

En cuanto a cadenas de texto (*string*), también existen muchos métodos que sirven para la manipulación de este tipo de datos, por ejemplo: **len(), lower(), translate(), islower(), upper(), startswith(), isupper(), find(), endswith(), isnumeric(), center(), rfind(), isalnum(), isdecimal(), isalpha(), split(), findall(), replace(), contains(), cat(), nomalize()**, etc.(VanderPlas, 2016).

Matplotlib

Es una librería multiplataforma para visualización de datos construida sobre arreglos NumPy. Una de sus características más importantes es la habilidad de trabajar con muchos sistemas gráficos y operativos de *backend*.

Como convención se suele utilizar la librería Matplotlib con el nombre corto **mpl** y su método más usado **pyplot** con el nombre corto **plt** (Matplotlib, 2023):

import matplotlib as mpl

import matplotlib.pyplot as plt

Para guardar figuras en una amplia gama de formatos se utiliza el método **savefig()** y como parámetros se envían el *path* en donde se quiera guardar (por defecto, si no se especifica, se toma el directorio actual), el nombre con el que se quiera guardar y su formato correspondiente.

Cuenta con dos posibles interfaces de uso, el resultado de un ejemplo para ambos es igual y se presenta en la Ilustración 2:

Estilo MATLAB: ya que Matplotlib fue originalmente desarrollada como una alternativa a MATLAB, esto se refleja en su sintaxis. Las herramientas de esta interfaz están contenidas en el método **pyplot** (**plt**) y todos los comandos que se usen mediante esta interfaz hacen referencia a la figura actual. Esta interfaz es suficientemente útil para gráficos relativamente sencillos y que no contengan muchos elementos o características.

 # Crea el primer subgráfico

plt.subplot(2, 1, 1) # (filas, columnas, num de subgráfico)

plt.plot(x, np.sin(x)) # gráfico (sobre la figura actual)

 # Crea el segundo subgráfico

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

plt.subplot(2, 1, 2) # (filas, columnas, num de subgráfico)

plt.plot(x, np.cos(x)) # gráfico (sobre la figura actual)

Orientada a objetos: Esta interfaz está disponible para situaciones más complejas en donde es necesario un mayor control sobre la figura. En esta interfaz las funciones de graficado son métodos de objetos explícitos.

 # Primero crea la malla de subgráficos

 # ax es un arreglo de dos objetos

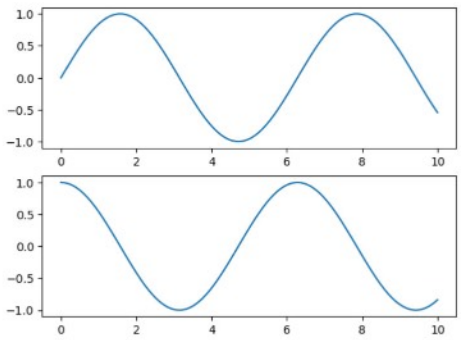
fig, ax = plt.subplots(2)

 # Llama el método plot() sobre el objeto apropiado

ax[0].plot(x, np.sin(x))

ax[1].plot(x, np.cos(x))

ILUSTRACIÓN 2. EJEMPLO DE FIGURA QUE CONTIENE DOS SUBGRÁFICAS, GENERADA CON MATPLOTLIB.



Existen muchos parámetros o propiedades para poder personalizar ampliamente las gráficas generadas, como el color (**color**), tipo de línea (**linestyle**), ancho de línea (**linewidth**), marcador (**marker**) y tamaño del marcador (**markersize**), entre otros. Y de igual manera, existen muchos métodos para personalizar los elementos de la gráfica, como tamaños, límites (**axis()**, **xlim()**, **ylim()**) y nombres (**xlabel()**, **ylabel()**) de los ejes, título (**title()**), leyenda (**legend()**), etc.

En la mayoría de los casos, la diferencia entre ambas interfaces mencionadas anteriormente es mínima, y basta con cambiar el objeto de referencia entre plt y ax. Sin embargo, existen también casos en los que los métodos cambian de nombre.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Estos cambios se presentan en la Tabla 5. En la interfaz orientada a objetos es mucho más conveniente utilizar el método ax.set() para definir todas estas propiedades a la vez, en lugar de irlas modificando una a una, y en donde si se mantiene el formato de los nombres (VanderPlas, 2016).

TABLA 5. COMPARACIÓN DE MÉTODOS ENTRE INTERFACES DE MATPLOTLIB

Interfaz MATLAB	Interfaz orientada a objetos
plt.xlabel()	ax.set_xlabel()
plt.ylabel()	ax.set_ylabel()
plt.xlim()	ax.set_xlim()
plt.ylim()	ax.set_ylim()
plt.title()	ax.set_title()

También existen más métodos para diferentes tipos de gráficas, por ejemplo:

Gráficos de dispersión (**scatter()**): cuyo funcionamiento es bastante similar a **plot()**, con algunos parámetros adicionales, como barra de colores y opacidad para los puntos, etc. Hay una consideración en cuanto a eficiencia y optimización: para pequeñas cantidades de datos a graficar no existe mayor diferencia en el desempeño, sin embargo, para cantidades grandes (alrededor de los miles) **plt.scatter()** es menos eficiente debido al proceso de renderizado (tamaño y color) para cada punto.

Gráficas de errores (**errorbar()**): que sirve para representar datos que contengan un margen de error. De igual manera, tiene parámetros para personalización como marcadores y opacidad, atributos especialmente útiles para conjuntos de datos extensos.

Gráficas con errores continuos (**fill()** y **fill_between()**): que aplica un proceso de regresión gaussina – GPR.

Gráficas en 3D (**contour()**, **contourf()**, **plot3D()**, **scatter3D()**, **contour3D()**): que permiten representar datos en 3D sobre una gráfica en 2D mediante contornos y colores, o propiamente en una 3D.

Gráficos estadísticos (**hist()**, **hist2d()**, **hexbin()**): que sirven para graficar histogramas tradicionales y en 2D.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

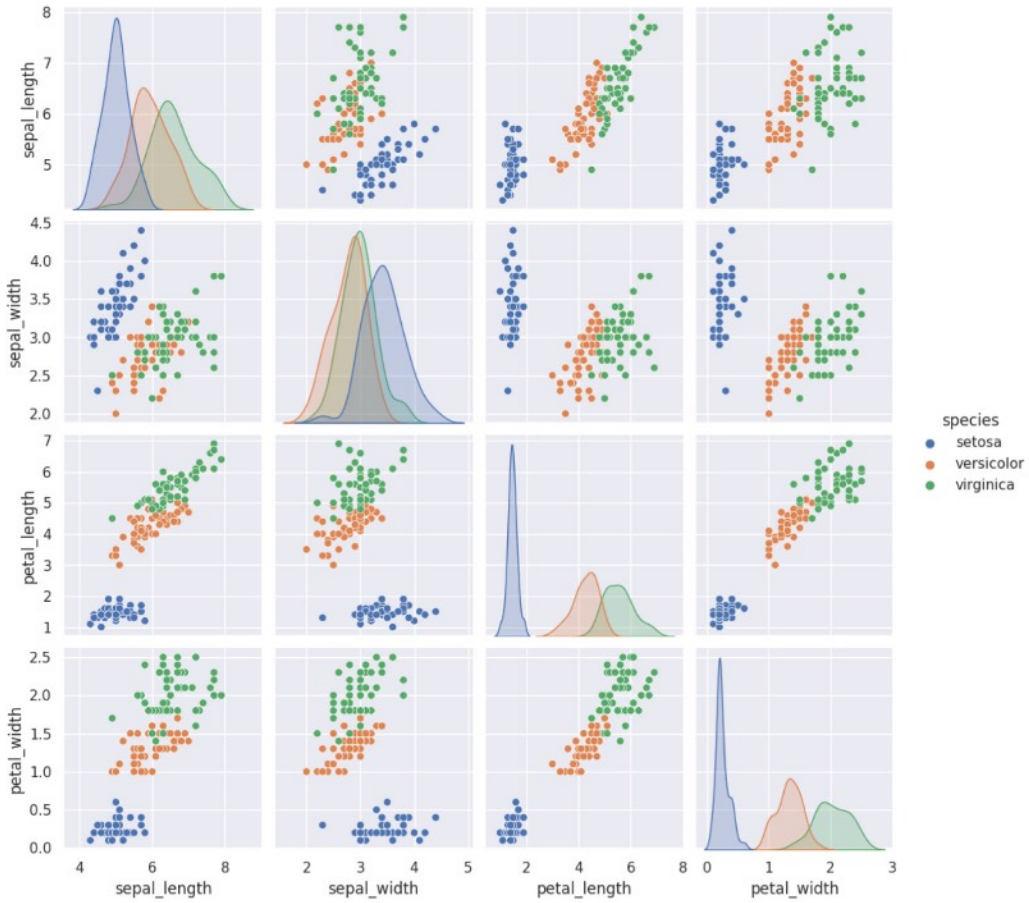
Seaborn

Seaborn (Seaborn, 2023) proporciona una API que funciona sobre Matplotlib, y que ofrece una visualización más profesional de las gráficas, manteniendo los mismos métodos y parámetros usando su método **set()**. Además, a diferencia de Matplotlib, está diseñado para trabajar con objetos DataFrame de Pandas. Como convención se suele utilizar la librería con el nombre corto **sns**:

import seaborn as sns

Adicionalmente, incorpora un método bastante útil para un primer acercamiento a datos con muchas dimensiones o características (*features*). Este método **pairplot()**, presenta una matriz con una comparación visual entre pares de datos y en su diagonal presenta los histogramas correspondientes de dichos datos (VanderPlas, 2016). En la Ilustración 3 se presenta un ejemplo de este método aplicado al conjunto de datos *iris*.

ILUSTRACIÓN 3. EJEMPLO DEL MÉTODO PAIRPLOT() DE SEABORN A PARTIR DEL DATASET "IRIS"



Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Conclusiones

En conclusión, el manejo de datos en Python es una habilidad esencial para cualquier persona que trabaje con datos, ya sea en el ámbito académico, empresarial o personal. Python es un lenguaje de programación muy popular y versátil que ofrece una gran cantidad de herramientas y librerías para el manejo de datos.

En esta lectura de profundización, se presentó cómo almacenar y manipular datos en Python, desde la estructura básica de datos y listas hasta el uso de librerías especializadas como NumPy, Pandas, Matplotlib y Seaborn. También se presentó cómo importar estas librerías en el ambiente personal de trabajo y cómo utilizarlas para realizar operaciones matemáticas, estadísticas y de visualización de datos.

Es importante destacar que el manejo de datos en Python no se limita a la manipulación de datos numéricos, sino que también incluye la manipulación de datos de texto, imágenes, audio y video, entre otros. Python es un lenguaje muy flexible y adaptable que se puede utilizar en una gran variedad de aplicaciones y proyectos.

En resumen, el manejo de datos en Python es una habilidad esencial para cualquier persona que trabaje con datos. Python ofrece una gran cantidad de herramientas y librerías para el manejo de datos, desde las estructuras básicas de las listas hasta las librerías especializadas como NumPy, Pandas, Matplotlib y Seaborn. Además, el manejo de datos en Python no se limita a la programación, sino que también incluye el uso de herramientas de análisis y visualización de datos.

Desde luego, este contenido presenta información esencial y reducida de todos los métodos y parámetros disponibles para cada librería mencionada. Si se quiere profundizar en estos temas se debe acudir directamente a la documentación oficial de cada librería y métodos, disponible en las *webs* oficiales. De igual manera, muchas implementaciones o usos de estas librerías son bastante específicas y se logran mediante experiencia, algunas veces en base a prueba-error, en donde se deben explorar diversos acercamientos para obtener el resultado deseado.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Referencias

Bisong, E. (2019). Matplotlib and Seaborn. En E. Bisong (Ed.), *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners* (pp. 151–165). Apress. https://doi.org/10.1007/978-1-4842-4470-8_12

Chen, D. Y. (2017). *Pandas for Everyone: Python Data Analysis.* Addison-Wesley Professional.

Matplotlib. (2023). *Pyplot tutorial* – Matplotlib 3.8.1 documentation. https://matplotlib.org/stable/tutorials/pyplot.html

Numpy. (2022). *NumPy documentation* – NumPy v1.26 *Manual.* https://numpy.org/doc/stable/

Pandas. (2023). P*andas documentation* — Pandas 2.1.2 documentation. https://pandas.pydata.org/docs/

Seaborn. (2023). *User guide and tutorial* — Seaborn 0.13.0 documentation. https://seaborn.pydata.org/tutorial.html

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data.* O'Reilly Media, Inc.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.